

S-band TLM 処理・DB 格納・HTML Viewer 試験マニュアル

2026 年 6 月 5 日

1 目的

本書は、s_band フォルダに含まれる試験用ツール一式の使い方をまとめる。対象は、S-band TLM ログを processor で処理し、パケット単位のデータを Main HK と ADCS HK の 2 系統に分けて DB サーバへ格納し、HTML viewer から検索・確認・CSV ダウンロードする試験手順である。

2 フォルダ構成

フォルダ	役割
processor/	TLM ログの ingest、CRC 検証、パケット解析、デコード可能 payload の構築、DB アップロードを行う。
server/	試験用 FastAPI DB サーバ。SQLite に payload 行を保存し、検索 API と CSV ダウンロード API を提供する。製品版ではなく開発・試験用。
downloader/	ブラウザで開く HTML viewer。DB サーバに接続し、Main HK / ADCS HK を検索・プレビュー・CSV ダウンロードする。
decoder_core/	processor と server が共有するデコーダ群。最新版と日付付きアーカイブ版を管理する。

3 全体フロー

1. DB サーバをローカルで起動する。
2. HTML viewer をローカルで起動し、DB サーバへの接続状態を確認する。
3. TLM テキストログを processor/input/unprocessed/ に配置する。
4. processor を実行する。
5. processor が TLM をパケット候補に分割し、CRC 正常なパケットだけを残す。
6. パケット構造 JSON に従って、Packet ID、Packet no.、Datetime、Data などの行に変換する。
7. Packet ID ごとに payload を再構成し、DB 格納対象を Main HK と ADCS HK に分ける。
8. DB サーバへ HTTP POST で格納する。
9. HTML viewer から検索、プレビュー、CSV ダウンロードを行う。

4 Python 環境の方針

本ツール群では、トップフォルダに共通 venv を作るのではなく、各ソフトのフォルダ配下に個別の .venv と requirements.txt を置く。実行時も、原則として対象ソフトのフォルダへ移動してから、そのフォルダの venv を有効化する。これは、設定ファイル、入力ファイル、SQLite DB などの相対パスが各ソフトのフォルダ基準になっているためである。

ソフト	環境	備考
processor/	.venv, requirements.txt	require- TLM 処理、pytest、pandas、crcmod などを含む。
server/	.venv, requirements.txt	require- FastAPI、uvicorn、pandas を含む。DB パスは server フォルダ基準。
downloader/	.venv, requirements.txt	require- 外部 Python package は不要。標準ライブラリの http.server で静的 viewer を配信する。
decoder_core/	.venv, requirements.txt	require- 共有デコーダ単体確認用。pandas、pytz を含む。

4.1 環境作成・更新コマンド

各フォルダで以下を実行する。

```
cd processor
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt

cd server
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt

cd downloader
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt

cd decoder_core
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
```

5 processor の処理概要

processor の主な開発用入口は processor/dev/run_pipeline.py である。簡易実行用に processor/run も用意されている。

5.1 処理段階

段階	内容
ingest_raw_log	生 TLM テキストを読み込み、ログ中の時刻と 16 進データを正規化し、タイムスタンプ付き binary を作る。
verify_crc	sync code を基準にパケット候補を切り出し、CRC が正しいパケットだけを結合して残す。
parse_packets	GSE 種別に応じた packet structure JSON を使い、binary パケットを DataFrame 行へ変換する。
build_decodable_payloads	Packet ID ごとに並べ替え、欠番を検出し、デコード可能な payload chunk を構築する。
decode_payloads	共有デコーダを使って decoded CSV を出力する。

5.2 DB 格納時の 2 系統分割

DB 格納・アップロード時は、Packet ID の先頭 3 bit を data type として扱う。

先頭 3 bit	系統	DB/API
110	Main HK	main_hk_payloads、/payloads/main-hk
011	ADCS HK high	adcs_hk_payloads、/payloads/adcs-hk
100	ADCS HK normal	adcs_hk_payloads、/payloads/adcs-hk

Main HK は OBC timestamp を持つ行として保存される。ADCS HK は ADCS timestamp と sampling type を持つ行として保存される。

6 DB サーバの起動

6.1 依存関係のインストール

```
cd server
.venv/bin/python -m pip install -r requirements.txt
```

6.2 ローカル起動

```
cd server
.venv/bin/python -m uvicorn app:app --host 127.0.0.1 --port 8000 --reload
```

起動後、以下を開く。

<http://127.0.0.1:8000/docs>

DB の既定パスは、server フォルダから起動した場合、次である。

server/data/payloads.sqlite

6.3 主な API

API	用途
/health	サーバ状態と DB パスを確認する。
/decoders	viewer の decoder 選択肢を取得する。
/main-hk	Main HK payload の JSON 検索・プレビュー。
/adcs-hk	ADCS HK payload の JSON 検索・プレビュー。
/downloads/main-hk.csv	Main HK CSV ダウンロード。
/downloads/adcs-hk.csv	ADCS HK CSV ダウンロード。
/payloads/main-hk	processor から Main HK payload を POST する。
/payloads/adcs-hk	processor から ADCS HK payload を POST する。

7 HTML viewer の起動

7.1 起動

```
cd downloader
.venv/bin/python -m http.server 8080
```

ブラウザで以下を開く。

http://127.0.0.1:8080

画面右上の API 欄の既定値は次である。

http://127.0.0.1:8000

DB サーバが起動していれば、画面上部に Connected と DB パスが表示される。

7.2 viewer の機能

- Main HK タブで OBC 時刻範囲、受信時刻範囲、GSE、件数、並び順、decoder を指定して検索する。
- ADCS タブで ADCS 時刻範囲、受信時刻範囲、GSE、sampling type、件数、並び順、decoder を指定して検索する。
- Search でプレビューを表示する。
- Download CSV で CSV を取得する。
- Raw download を有効にすると、保存済み raw payload 行をそのまま取得する。

8 TLM ログの処理

8.1 入力ファイル配置

処理対象の .txt TLM ログを以下に置く。

```
processor/input/unprocessed/
```

8.2 全ファイルを処理

```
cd processor
.venv/bin/python ./run
```

input/unprocessed/ 内の .txt が処理され、成功したファイルは input/processed/ に移動される。

8.3 1 ファイルだけ処理

```
cd processor
.venv/bin/python ./run input/unprocessed/example.txt
```

8.4 入力ファイルを移動せずに処理

開発・再現確認では、入力ファイルを残すために --no-move を付ける。

```
cd processor
.venv/bin/python ./run input/unprocessed/example.txt --no-move
```

8.5 DB サーバ指定

既定のアップロード先は次である。

```
http://127.0.0.1:8000
```

明示する場合は環境変数またはオプションを使う。

```
cd processor
S_BAND_DECODER_DB_SERVER=http://127.0.0.1:8000 .venv/bin/python ./run

cd processor
.venv/bin/python ./run input/unprocessed/example.txt \
--db-server http://127.0.0.1:8000
```

8.6 DB へ送らずに処理

```
cd processor
.venv/bin/python ./run input/unprocessed/example.txt --no-db
```

8.7 ローカル SQLite に直接書き込む

HTTP サーバを使わず、processor 側の SQLite に直接保存する場合は `--local-db` を使う。

```
cd processor
.venv/bin/python ./run input/unprocessed/example.txt --local-db
```

9 出力先

パス	内容
processor/output/intermediate/	各処理段階の中間成果物。
processor/output/decoded/	decode_payloads による decoded CSV。
processor/output/pending_uploads/	DB サーバ接続失敗時の保留アップロード JSON。
processor/output/reports.jsonl	ingest 時の補正や検出結果などのレポート。
server/data/payloads.sqlite	試験用 DB サーバの SQLite DB。

10 DB サーバ停止中の扱い

processor 実行時に DB サーバへ接続できない場合、payload は破棄されず、次のフォルダに JSON として退避される。

```
processor/output/pending_uploads/
```

DB サーバ起動後、以下で再送する。

```
cd processor
.venv/bin/python -m dev.retry_uploads
```

サーバ URL を指定する場合は次のようにする。

```
cd processor
.venv/bin/python -m dev.retry_uploads --db-server http://127.0.0.1:8000
```

11 動作確認

11.1 DB サーバ確認

```
curl http://127.0.0.1:8000/health
```

11.2 テーブル確認

```
curl http://127.0.0.1:8000/tables
```

11.3 Main HK プレビュー

```
curl "http://127.0.0.1:8000/main-hk?limit=5"
```

11.4 ADCS HK プレビュー

```
curl "http://127.0.0.1:8000/adcs-hk?limit=5"
```

12 簡易実行フォルダ

トップフォルダ直下の `easy_run/` は、初心者向けに 4 ステップで使うための簡易実行フォルダである。エイリアスは後で作り直す前提のため、現時点では `00_TLM_INPUT_ALIAS_PLACEHOLDER.txt` を置いている。運用時はこのプレースホルダを、`processor/input/unprocessed/` へのエイリアスまたはショートカットに置き換える。

12.1 初回だけ実行

最初に 1 回だけ、`easy_run/00_setup_all` を実行する。このファイルは、`processor/`、`server/`、`downloader/`、`decoder_core/` の各 `.venv` を作成し、各 `requirements.txt` をインストールする。既に `.venv` がある場合は再利用する。

- Windows: `easy_run/00_setup_all.bat`
- macOS: `easy_run/00_setup_all.command`

12.2 毎回の 4 ステップ

1. TLM 投入エイリアスへ `.txt` ファイルを入れる。
2. `easy_run/01_start_server` を実行し、DB サーバを起動したままにする。
3. `easy_run/02_run_processor` を実行し、TLM を処理して DB へ格納する。
4. `downloader/index.html` を開き、viewer で Search を押す。

Windows では `.bat`、macOS では `.command` を使う。viewer を開くための補助ファイルとして、`easy_run/03_open_viewer` も用意している。

13 よく使う試験手順

1. Terminal 1 で DB サーバを起動する。
2. Terminal 2 で HTML viewer を起動する。
3. viewer を開き、Connected と表示されることを確認する。
4. TLM ログを `processor/input/unprocessed/` に置く。
5. Terminal 3 で `cdprocessor&&.venv/bin/python./run--no-move` を実行する。
6. viewer の Main HK / ADCS タブで Search を押し、行が増えていることを確認する。
7. 必要に応じて Raw download または decoded CSV をダウンロードする。

14 注意事項

- GSE はファイル名に `RX_COM` を含む場合 Kyutech、それ以外は ISAS と推定される。明示する場合は `--gseKyutech` または `--gseISAS` を指定する。
- server は試験用であり、README にも製品版では使用しない旨が記載されている。
- `server/app.py` にも `/downloader` があるが、実際の検索・ダウンロード用 viewer は `downloader/index.html` と `downloader/app.js` である。
- decoded download は最新版 decoder が既定である。日付付き decoder は選択肢として表示されるが、バージョン指定再デコードは未実装の扱いである。